Escuela Técnica Superior de Ingeniería

Universidad de Huelva

Grado en Ingeniería Informática

Programación de Juegos

Práctica 1: Videojuego 2D

Jose Manuel Moriana Herrera

Alejandro Martín Correa

ÍNDICE

1. Introducción 6
1.1. Descripción General 6
1.2. Controles 6
1.3. Lenguaje y motor de videojuegos 6
2. Arquitectura y Flujo del Juego 7
2.1. Escenas 7
2.1.1. MenuPrincipal 7
2.1.2. proyecto 10
3. Scripts 19
3.1. AplicarVolumen 19
3.2. CamaraManagement 19
3.3. FondoManagement 20
3.4. RanaMovement 21
3.5. DatosJuego 23
3.6. ControladorDatosJuego 24
3.7. DatosRanking 27
3.8. GestorRankings 27
3.9. VisualizadorRankingsUI 29
3.9. VidaJugador 29
3.10. SETA_PATROL 32
3.11. ABEJA_PATROL 34
3.12. Moneda 37
3.13. Trampoline 38
3.14. Teletransporte 39
3.15. Puntuaje 40
3.16. Cronometro 41
3.17. MENUINICIAL 42
3.18. CARGAR_PARTIDA 44
3.19. MenuGanar 44
3.20. MenuMorir 46
3.21. MenuPausa 48

4. Persistencia de Datos y Preferencias 52
5. Técnica de inteligencia artificial 52
6. Assets utilizados 58
7. Bibliografía 58

ÍNDICE DE ILUSTRACIONES

Ilustración 1: MenuInicial 7
Ilustración 2: MenuOpciones 8
Ilustración 3: MenuCarga 8
Ilustración 4: MenuRankings 9
Ilustración 5: MenuSonido 9
Ilustración 6: AudioMixer 10
Ilustración 7: RANA 10
Ilustración 8: Parallax 11
Ilustración 9: BotonPausa 12
Ilustración 10: MPausa 12
Ilustración 11: TiempoYPuntos 12
Ilustración 12: MMorir 13
Ilustración 13: MGanar 13
Ilustración 14: Grid 14
Ilustración 15: TELEPORT 14
Ilustración 16: FINISH_FLAG 15
Ilustración 17: VIDAS 15
Ilustración 18: Moneda 16
Ilustración 19: Trampolin 16
Ilustración 20: ENEMIGO_SETA 17
Ilustración 21: ABEJA 18
Ilustración 22: AplicarVolumen 19
Ilustración 23: CamaraManagement 20
Ilustración 24: FondoManagement 20
Ilustración 25: RanaMovement 21
Ilustración 26: AnimatorRana 22
Ilustración 27: RanaMovement 2 22
Ilustración 28: RanaMovement 3 23
Ilustración 29: DatosJuego 23
Ilustración 30: ControladorDatosJuego 24
Ilustración 31: ControladorDatosJuego 2 25

Ilustración 32: CargarDatos 25
Ilustración 33: GuardarDatos 26
Ilustración 34: BorrarPartidaActual 26
Ilustración 35: DatosRanking 27
Ilustración 36: GestorRankings 27
Ilustración 37: GestorRankings 2 28
Ilustración 38: ObtenerTextoFormateado 28
Ilustración 39: VisualizadorRankingsUI 29
Ilustración 40: VidaJugador 30
Ilustración 41: VidaJugador 2 30
Ilustración 42: VidaJugador 3 31
Ilustración 43: VidaJugador 4 32
Ilustración 44: SETA_PATROL 33
Ilustración 45: AnimatorSeta 33
Ilustración 46: PuntosSeta 33
Ilustración 47: SETA_PATROL 2 34
Ilustración 48: Variables ABEJA_PATROL 35
Ilustración 49: Start ABEJA_PATROL 35
Ilustración 50: ABEJA_PATROL 36
Ilustración 51: OnDrawGizmosSelected 36
Ilustración 52. Moneda 1. 37
Ilustración 53. Moneda 2. 37
Ilustración 54. Moneda 3 38
Ilustración 55.Trampolín 1 38
Ilustración 56.Trampolín 2 39
Ilustración 57. Teletransporte 40
Ilustración 58.Puntuaje 1 40
Ilustración 59. Puntuaje 2 41
Ilustración 60. Puntuaje 3 41
Ilustración 61. Cronómetro 42
Ilustración 62.MenuInicial 1 42
Ilustración 63. MenuInicial 2 43
Ilustración 64.MenuInicial 3 43
Ilustración 65.CARGAR_PARTIDA 44
Ilustración 66.MenuGanar 1 45
Ilustración 67. MenuGanar 2 46
Ilustración 68. MenuGanar 3 46
Ilustración 69. MenuMorir 1 47
Ilustración 70. MenuMorir 2 47
Ilustración 71. MenuPausa 1 48
Ilustración 72. MenuPausa 2 49

Ilustración 73. MenuPausa 2 50
Ilustración 74. MenuPausa 3 51
Ilustración 75. Enum EstadoAbeja 52
Ilustración 76. Diagrama de estados 52
Ilustración 77. Estados Abeja 1 54
Ilustración 78. Estados Abeja 2 54
Ilustración 79. Patrullar 55
Ilustración 80. Perseguir 56
Ilustración 81. Atacar 56
Ilustración 82. Retirarse 57
Ilustración 83. CambiarEstado 57

1. Introducción

1.1. Descripción General

Jumping frog es un videojuego 2D del género plataformas. Tiene una temática alegre, dado que predominan colores vivos y saturados.

El objetivo del juego es llevar al jugador hacia la meta en el menor tiempo posible, también se tendrán en cuenta los puntos obtenidos y las vidas con las que se llega a la meta. Al finalizar el juego el jugador puede ver su posición en el ranking.

1.2. Controles

La jugabilidad es muy sencilla, se utilizan tanto teclado como ratón. Para el movimiento de nuestro personaje se utilizan las teclas 'A' y 'D' para izquierda y derecha, respectivamente. Para el salto, se utiliza la tecla 'espacio' y para abrir el menú de pausa, la tecla 'escape' o clic sobre el botón de pausa del HUD del juego. Mediante el indicador del ratón, el jugador se moverá por los menús del juego clicando sobre los botones existentes.

1.3. Lenguaje y motor de videojuegos

Se ha utilizado el lenguaje de programación C#, ya que es el que incorpora el motor de videojuegos Unity. En este caso, el motor utilizado es Unity en la versión 6000.4.0f1. Este lenguaje implementa la programación orientada a objetos, por lo tanto, se adapta a la arquitectura basada en componentes (GameObjects) que usaremos para su desarrollo.

2. Arquitectura y Flujo del Juego

2.1. Escenas

El juego consta de dos escenas, ‘MenuPrincipal’ y ‘proyecto’. Veamos que contiene cada escena:

2.1.1. MenuPrincipal

El menú inicial está compuesto por los botones de jugar, opciones y ranking. Cada botón engloba uno de los distintos submenús: cargado de partidas, sonido y rankings, respectivamente. La jerarquía de la escena es la siguiente:

- Main Camera: objeto de tipo Camera que posibilita la visualización de la escena.

- Canvas: objeto de tipo Canvas que contiene el script ‘MENUINICIAL.cs’ y los distintos tipos de menús existentes en la escena. Se escala con el tamaño de pantalla con una referencia de resolución de 1920x1080.
    - MenuInicial: objeto de tipo Image que contiene los botones iniciales de nuestro juego y el texto de inicio. Estos botones son: BOTON_JUGAR, BOTON_RANKINGS, BOTON_SALIR y BOTON_MENU. Cada botón tiene un texto TextMeshPro y un evento onClick().

Ilustración 1: MenuInicial

- MenuOpciones: objeto de tipo Image que contiene los botones EFECTOS_SONIDO y VOLVER, junto al título del juego. Al igual, que todos los botones utilizados, cada botón tiene un texto TextMeshPro y un evento onClick().

Ilustración 2: MenuOpciones

- MenuCarga: objeto de tipo Image que contiene los botones de las distintas partidas posibles a seleccionar y el botón de volver atrás, junto al título de nuestro juego. Cada botón está enlazado a un archivo "*.json*" donde se guardarán los correspondientes datos del jugador y el nivel.

Ilustración 3: MenuCarga

- MenuRankings: objeto de tipo Image que contiene los botones BOTON_RANKINGS y VOLVER_MENU, junto al título del juego. Aquí se muestra el ranking de las 10 mejores marcas que se ha logrado en el nivel.

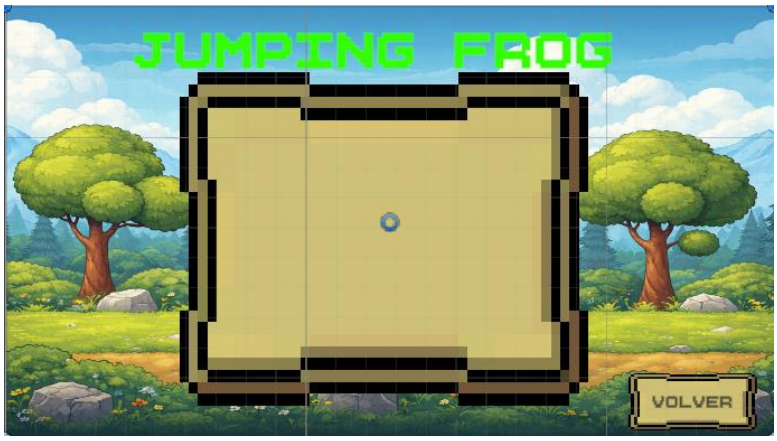

Ilustración 4: MenuRankings

- MenuSonido: objeto de tipo Image que contiene los sliders SLIDER_MUSICA y SLIDER_SFX y el botón de volver atrás. SLIDER_SFX maneja el volumen de los efectos visuales dentro del juego y SLIDER_MUSICA maneja el volumen de la música tanto del menú principal como de la música dentro de la partida.

Ilustración 5: MenuSonido

– AUDIO_MENU: objeto de tipo Audio Source que contiene la música del menú principal y el script 'AplicarVolumen.cs'. Se hace uso de un Audio Mixer definido como 'AUDIO_MENU', concretamente el subgrupo Volumen como salida del audio.

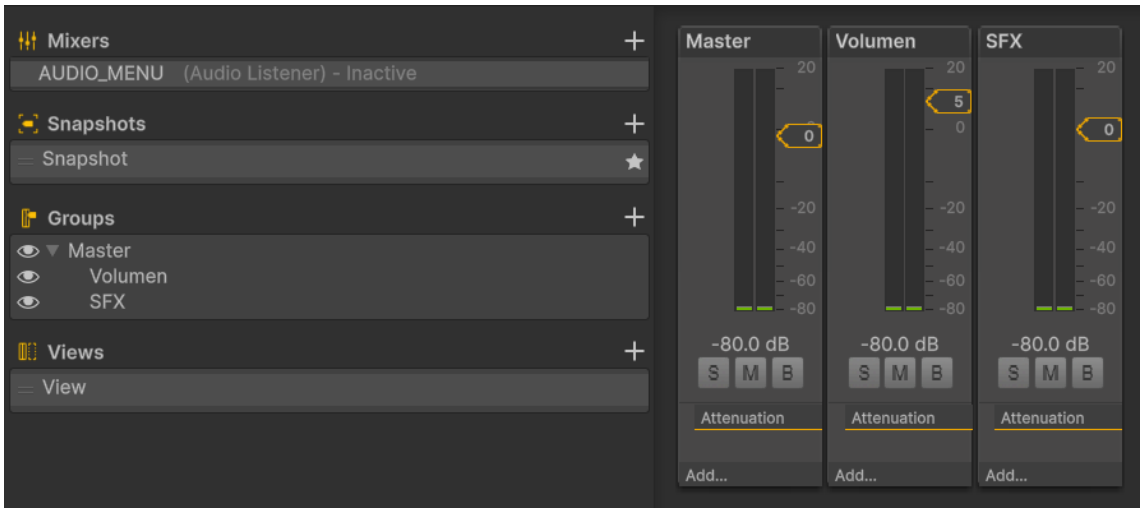

Ilustración 6: AudioMixer

- GestorRankings: objeto vacío que contiene el script 'GestorRanking.cs' para poder acceder a los datos necesarios para el ranking.

2.1.2. proyecto

La escena proyecto es la que contiene todos objetos necesarios para la partida: enemigos, interactuables, personaje, fondo y HUD. Veamos paso a paso cada uno de los objetos contenidos en esta jerarquía:

- RANA: objeto con el tag Player con el sprite del personaje principal. Dentro de sus propiedades tiene asociado un Rigidbody 2D de tipo dinámico con el material NoFriction (friction a 0 y bounciness a 0 para evitar que el personaje se adhiera a las paredes y se deslice), Capsule Collider 2D con material NoFriction y la opción isTrigger desactivada para que el personaje no atraviese el terreno de juego. También tiene asociado un Animator con dos estados Idle y Running, y dos scripts 'RanaMovement.cs' y 'VidaJugador.cs', entre ambos scripts se tienen en cuenta aspectos como límite de altura máxima y mínima a la que el jugador puede llegar, su velocidad, su fuerza de salto, los sonidos tanto de salto, victoria, muerte, música de fondo o cantidad de vidas, entre otros.

Ilustración 7: RANA

- FONDO_BOSQUE: objeto vacío con el tag Fondo que contiene un sorting group para poder ordenar las distintas partes del fondo en capas distintas, consiguiendo de esta forma un efecto 'parallax' en el fondo. Dentro de FONDO_BOSQUE se encuentran 7 trozos de fondo que forman el fondo completo, cada uno de ellos se encuentra en una capa distinta del sorting group y tiene asociado el script 'FondoManagement.cs' que configura la velocidad a la que se mueve cada capa del fondo. Dentro de este objeto en la jerarquía se encuentran:

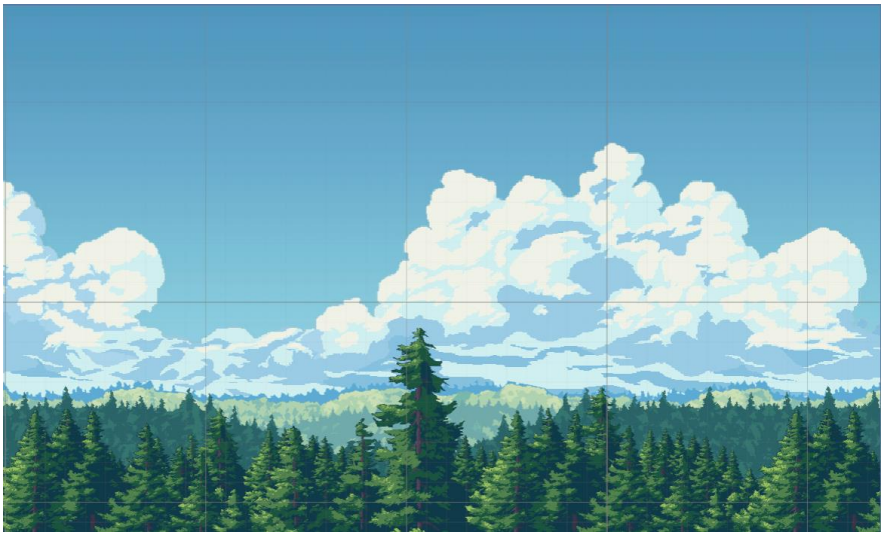

Ilustración 8: Parallax

  - Main Camera: objeto de tipo Camera con el script 'CamaraManagement.cs' asociado para poder seguir al personaje y mover el fondo a su vez.

  - PAUSA: objeto tipo Canvas con el render mode en Screen Space – Camera para mantenerlo en el HUD. Además, tiene asociado el script 'MenuPausa.cs' que maneja el botón de pausa (BotonPausa) y el menú de pausa dentro del nivel (MPausa). Al presionar el botón de Escape o el que gráficamente se muestra por pantalla aparece el menú de pausa, donde se podrá guardar partida, reanudar o salir al menú principal.

    - BotonPausa: objeto Button con evento onClick() mediante el script de PAUSA.

    - MPausa: objeto Image que tiene como hijos en la jerarquía los botones Reanudar, Guardar y VOLVER_MENU.

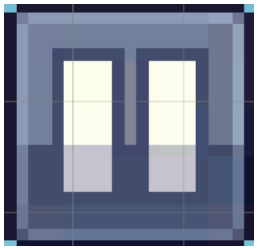

Ilustración 9: BotonPausa

Ilustración 10: MPausa

- Puntos: objeto tipo Canvas con el render mode en Screen Space – Camera para mantenerlo en el HUD. Tiene como hijo en la jerarquía un objeto TextMeshPro Puntuacion que muestra los puntos que vamos teniendo a lo largo de la partida. Puntuacion tiene asociado el script 'Puntuaje.cs'.

- Tiempo: objeto tipo Canvas con el render mode en Screen Space – Camera para mantenerlo en el HUD. Tiene como hijo en la jerarquía un objeto TextMeshPro tiempo que muestra el tiempo actual de la partida. tiempo tiene asociado el script 'Cronometro.cs'.

Ilustración 11: TiempoYPuntos

- MENU_MORIR: objeto tipo Canvas con el render mode en Screen Space – Camera y el script 'MenuMorir.cs' con el objeto MMorir asociado, hijo de MENU_MORIR en la jerarquía. MMorir es un objeto Image que tiene como hijos los botones REINTENTAR y VOLVER_MENU y el texto TextMeshPro que muestra los puntos con los que el personaje ha muerto.

Ilustración 12: MMorir

- MENU_GANAR: objeto tipo Canvas con el render mode en Screen Space – Camera y el script 'MenuGanar.cs' con el objeto MGanar asociado, hijo de MENU_GANAR en la jerarquía. MGanar es un objeto Image que tiene como hijos el botón VOLVER_MENU y el texto TextMeshPro que muestra los punto, vidas y tiempo con las que el jugador ha ganado.

Ilustración 13: MGanar

- FONDO_DESIERTO: al igual que FONDO_BOSQUE tiene varias capas, este objeto solo es utilizado para un pequeño ‘easter egg’ del juego.

- Grid: objeto de tipo Grid que contiene a su vez objetos de tipo Tilemap para renderizar los distintos sprites del terreno del juego con paletas: terreno y Winter. Estos tilemap tienen como propiedades un Composite Collider 2D, un Rigidbody 2D de tipo Static y un Tilemap Collider 2D con la operación Composite Merge para unificar todo los sprites del terreno del juego en un solo tile.

Ilustración 14: Grid

- TELEPORT: objeto con Sprite Renderer, un Animator que hace la animación del portal, un Box Collider 2D con is Trigger activo para que el personaje pueda atravesarlo y un script ‘Teletransporte.cs’ con el sonido del objeto asociado y el punto destino del teleporte. Este punto destino, TELEPORT_SITIO, es hijo dentro de la jerarquía del objeto TELEPORT y es un objeto vacío con las coordenadas del sitio donde el jugador será teletransportado cuando cruce el portal.

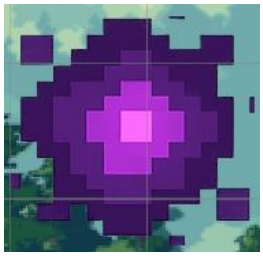

Ilustración 15: TELEPORT

- FINISH_FLAG: objeto con el tag Bandera, el sprite de la bandera de fin del juego al ganar, una pequeña animación mediante Animator y un Circle Collider 2D con is Trigger activo para que le personaje puede cruzarla.

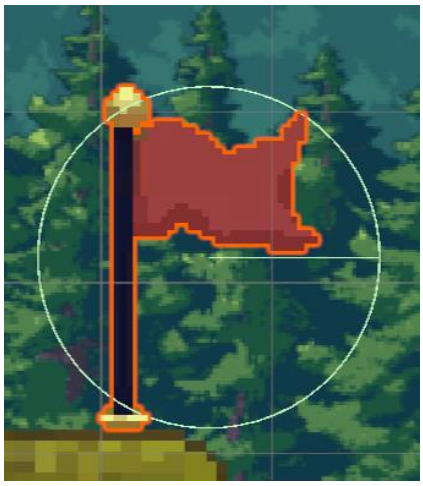

Ilustración 16: FINISH_FLAG

- VIDAS: objeto con el tag Vidas de tipo Canvas con el Render Mode en Screen Space – Camera para que sean fijas en el HUD y se muevan junto a la cámara. Dentro de la jerarquía tiene como hijos las distintas vidas como 'prefabs' del sprite del corazón utilizado. Para crear un prefab es necesario tenerlo como asset genérico y todos los sprites que utilicen este prefab tendrán las mismas características que tenga el prefab definido. En este caso, todos tienen como propiedad un Animator para la animación del HUD.

Ilustración 17: VIDAS

- Interactuables: objeto vacío que contiene todos los interactuables del juego, como monedas y trampolines.

    - Moneda: prefab Moneda con su sprite correspondiente y una animación. Además, como propiedades tiene un Circle Collider 2D isTrigger activo y un script 'Moneda.cs' con la cantidad de puntos que da cada Moneda, en este caso 50 puntos, y el efecto de sonido al interactuar con ella.

    - MonedaGrande: prefab identico al de Moneda, pero con sonido distinto y más potente con una cantidad de puntos mayor a las otras monedas.

    - Trampolín: prefab Trampolín con su sprite correspondiente, Animator, Box Collider 2D, script 'Trampoline.cs' que maneja la fuerza de salto y la animación, y un Audio Source con el sonido del trampolín con la salida de audio en el subgrupo SFX del Audio Mixer AUDIO_MENU. Permite realizar un salto más fuerte que el del jugador.

Ilustración 18: Moneda

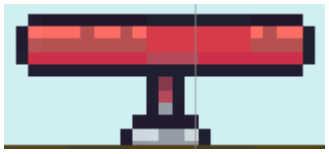

Ilustración 19: Trampolin

- GestorRankings: objeto vacío que contiene el script 'GestorRanking.cs' para poder guardar los datos necesarios para el ranking y se mantengan en el cambio de escena.

- Controlador: objeto vacío que contiene el script 'ControladorDatosJuego.cs' que mantiene datos del juego como, posición del jugador, vidas, fecha, puntuación, tiempo de juego.

- ENEMIGOS: objeto vacío que contiene como hijos en la jerarquía los distintos enemigos de la partida, ENEMIGO_SETA y ABEJAS.

    - ENEMIGO_SETA: objeto con el Tag Enemy con un Animator, Box Collider 2D, Rigidbody 2D Dynamic y el script 'SETA_PATROL.cs'. Este script controla los dos puntos por los que el enemigo se mueve (PUNTO_SETA_A, PUNTO_SETA_B) y la velocidad con lo que lo hace.

Ilustración 20: ENEMIGO_SETA

    - ABEJAS: objeto vacío que contiene como hijos los objetos Bee_0.

        - Bee_0: estos objetos sprites tienen asociados un Animator, un Rigidbody 2D Dynamic, Circle Collider 2D y un script 'ABEJA_PATROL.cs'. Este script, controla la velocidad de movimiento, el suavizado de movimiento, el cooldown de ataque, el tiempo y la distancia de retirada, la fuerza de evasión de obstáculos, el movimiento de patrulla de la abeja, los radios de los distintos estados de la abeja con respecto al jugador y el tiempo de decisión para elegir un mejor camino. Para los estados de la abeja se utiliza la técnica de IA, máquina de estados.

            - HITBOX: objeto con el tag EnemyAbeja con Circle Collider 2D isTrigger activado. Con este objeto se consigue que la animación de ataque de la abeja pueda golpear al jugador.

        - PUNTO: objetos vacíos con coordenadas para indicar la patrulla de la abeja. Cada abeja tiene asociado un número de puntos y se sigue un orden de puntos para patrullar.

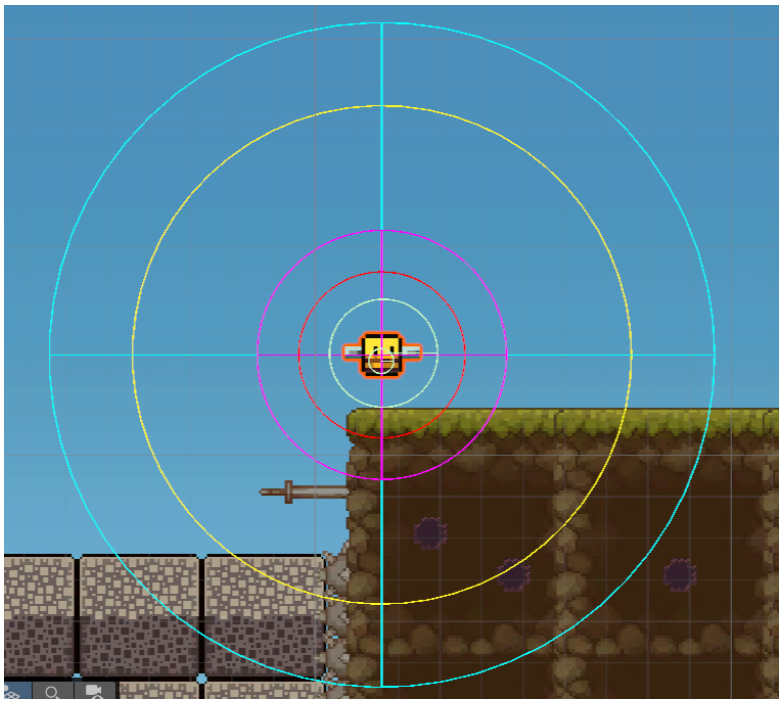

Ilustración 21: ABEJA

- OBJETOS_SONIDO: objeto vacío que contiene como hijos en la jerarquía los distintos Audio Source de los elementos del nivel.
    - AUDIO_NIVEL: Audio Source que tiene como salida el subgrupo Volumen del Audio Mixer AUDIO_MENU.
    - AUDIO_BICHO, AUDIO_SALTO, AUDIO_MONEDA, AUDIO_MONEDA_G, AUDIO_TRAMPOLIN, AUDIO_TRAMPOLIN, AUDIO_TELEPORT, AUDIO_BANDERA, AUDIO_PERDER: todos son Audio Source que tienen como salida el subgrupo SFX del Audio Mixer AUDIO_MENU.

3. Scripts

3.1. AplicarVolumen

El script 'AplicarVolumen.cs' controla el volumen de la música y los efectos de sonido con las preferencias que el jugador elige. Incluye un AudioMixer, referencia al mezclador de audio de Unity y usa PlayerPrefs para cargar los valores del sonido guardados.

En la función Start(), se obtienen los valores guardados para cada tipo de audio, estos valores se ponen en decibelios, rango de 0.0001 a 1. En caso de ser el valor menos de 0.0001, se ponen a –80db que es el mínimo (mute). Por último, se aplica el volumen a cada tipo de audio.

```
using UnityEngine;
using UnityEngine.Audio;

public class AplicarVolumen : MonoBehaviour
{
    [SerializeField] private AudioMixer audioMixer;

    void Start()
    {
        float valorM = PlayerPrefs.GetFloat("VOLUMEN", 1f);
        float valorE = PlayerPrefs.GetFloat("SFX", 1f);

        float dbM = valorM > 0.0001f ? Mathf.Log10(valorM) * 20 : -80f;
        float dbE = valorE > 0.0001f ? Mathf.Log10(valorE) * 20 : -80f;

        audioMixer.SetFloat("VOLUMEN", dbM);
        audioMixer.SetFloat("SFX", dbE);
    }
}
```

Ilustración 22: AplicarVolumen

3.2. CamaraManagement

Este script controla cómo la cámara sigue al jugador durante la partida. Tiene las variables player (objeto a seguir), offset (distancia entre la cámara y jugador), smoothSpeed (suavizado de la cámara), alturaMinima y elevacionAlCaer, para saber cuándo dejar de seguir al jugador, que en este caso es cada vez que muera por caída.

La función LateUpdate() se ejecuta tras actualizar la posición del jugador, desiredPosition guarda la posición del jugador más un pequeño offset. Si el jugador cae, la cámara no baja tanto y se limita al mínimo permitido. Para el movimiento suave de la cámara se utiliza smoothedPosition mediante una interpolación (Lerp), de esta forma se evitan movimientos bruscos. Por último, aplica la posición.

```
public class CamaraManagement : MonoBehaviour
{
    public Transform player;
    public float smoothSpeed = 5f;
    public Vector3 offset;

    public float alturaMinima = -7.23f;
    public float elevacionAlCaer = 1.5f;

    void LateUpdate()
    {
        if (player == null) return;

        Vector3 desiredPosition = player.position + offset;

        if (player.position.y < alturaMinima) //rana se cae
        {
            desiredPosition.y = alturaMinima + elevacionAlCaer;
        }
        else if (desiredPosition.y < alturaMinima)
        {
            desiredPosition.y = alturaMinima;
        }

        Vector3 smoothedPosition = Vector3.Lerp(transform.position, desiredPosition, smoothSpeed * Time.deltaTime);
        transform.position = smoothedPosition;
    }
}
```

Ilustración 23: CamaraManagement

3.3. FondoManagement

Este script crea el efecto parallax, el fondo se mueve más lento que el jugador. El script tiene como variables cam (referencia a la cámara), startpos (posición inicial), length (tamaño del fondo) y parallaxMultiplierX, velocidad a la que se mueve el fondo en un rango de 0 a 1, donde 0 significa quieto y 1 igual a la cámara. La función Start(), obtiene el valor inicial de los atributos anteriormente mencionados. La función Update() es la que logra el movimiento de cada sprite del fondo, cada uno de ellos a uno velocidad distinta, teniendo en cuenta si el movimiento de la cámara es a la izquierda o a la derecha y creando un fondo “infinito”.

```
public class FondoManagement : MonoBehaviour
{
    [Header("Configuración Parallax")]
    [Range(0f, 1f)]
    public float parallaxMultiplierX = 0.1f;

    private Transform cam;
    private Vector2 startpos;
    private float length;

    void Start()
    {
        cam = Camera.main.transform;

        // Guardamos la posición inicial
        startpos = new Vector2(transform.position.x, transform.position.y);
        length = GetComponent<SpriteRenderer>().bounds.size.x;
    }

    void Update()
    {
        // Movimiento relativo al efecto parallax (Solo X)
        float temp = (cam.position.x * (1 - parallaxMultiplierX));

        // Distancia que debe moverse en X
        float distX = (cam.position.x * parallaxMultiplierX);

        // Aplicamos el movimiento (startpos.y se queda fijo, no se le suma nada)
        transform.position = new Vector3(startpos.x + distX, startpos.y, transform.position.z);

        // Lógica de repetición infinita en el eje X
        if (temp > startpos.x + length)
        {
            startpos.x += length;
        }
        else if (temp < startpos.x - length)
        {
            startpos.x -= length;
        }
    }
```

Ilustración 24: FondoManagement

3.4. RanaMovement

El script 'RanaMovement.cs' controla el movimiento del personaje hacia derecha e izquierda, su velocidad, el salto, la detección de suelo, animaciones, sonidos y límite de alturas. Inicialmente en Start() se obtiene los componentes del personaje, inicializa los límites y prepara al sistema.

```
public class RanaMovement : MonoBehaviour
{
    public float velocidad;
    public float JumpForce;

    private Rigidbody2D rb;
    private float horizontal;
    private Animator animator;
    private bool grounded;
    private bool muerto = false;

    [Header("Límites")]
    [SerializeField] private float alturaMuerte;
    [SerializeField] private float alturaMaxima;


    [Header("SFX")]
    [SerializeField] private AudioSource audioSource;
    [SerializeField] private AudioClip sonidoSaltar;

    [SerializeField] private AudioSource audioSource2;
    [SerializeField] private AudioClip sonidoGanar;

    [SerializeField] private AudioSource audioSource3;
    [SerializeField] private AudioClip sonidoPerder;

    [Header("Música de Fondo")]
    [SerializeField] private AudioSource musicaFondo;

    void Start()
    {
        rb = GetComponent<Rigidbody2D>();
        animator = GetComponent<Animator>();
        alturaMaxima = 21f;
        alturaMuerte = -12f;
    }
```

Ilustración 25: RanaMovement

En la función Update(), que se ejecuta cada frame, se tiene en cuenta si se pulsa la tecla 'A' para ir hacia la izquierda o si se pulsa la tecla 'D' para ir hacia la derecha, detecta al suelo mediante DrawRay y Raycast lanzando un rayo hacia abajo y si toca algo, el personaje está en el suelo.

También se tiene en cuenta que si el jugador está en el suelo y pulsa la tecla 'space' se lanza la función Jump(), cuando está función se ejecuta se aplica el sonido de saltar y aplica una velocidad vertical al jugador.

Además, en Update(), se anima al jugador si está o no corriendo. Si se mueve, aplica la animación de correr, si no, está en la animación Idle, esto se controla mediante el booleano running. Para girar al personaje si cambia la coordenada de X de positivo a negativo dependiendo del valor de la variable horizontal.

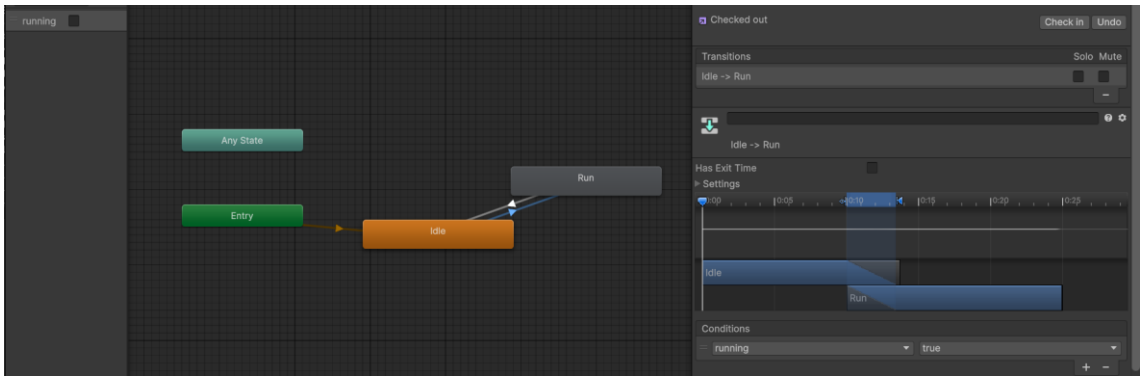

Ilustración 26: AnimatorRana

Por último, Update() detecta si el jugador muere por caída y lanza la función Morir() si la variable muerto está a false. Morir() busca el objeto MenuMorir en la jerarquía de la escena, reproduce el sonido de muerte y muestra el menú de morir mediante el objeto MenuMorir.

FixedUpdate() es usado para el movimiento horizontal y el límite superior.

```
void Update()
{
    horizontal = Keyboard.current.aKey.isPressed ? -1 :
                 Keyboard.current.dKey.isPressed ? 1 : 0;

    Debug.DrawRay(transform.position, Vector3.down * 0.6f, Color.red);
    grounded = Physics2D.Raycast(transform.position, Vector3.down, 0.6f);

    if (Keyboard.current.spaceKey.wasPressedThisFrame && grounded)
    {
        Jump();
    }

    animator.SetBool("running", horizontal != 0.0f);

    if (horizontal < 0.0f)
        transform.localScale = new Vector3(-1.0f, 1.0f, 1.0f);
    else if (horizontal > 0.0f)
        transform.localScale = new Vector3(1.0f, 1.0f, 1.0f);

    if (!muerto && transform.position.y < alturaMuerte)
    {
        muerto = true;
        Morir();
    }
}

void FixedUpdate()
{
    rb.linearVelocity = new Vector2(horizontal * velocidad, rb.linearVelocity.y);

    if (transform.position.y >= alturaMaxima)
    {
        if (rb.linearVelocity.y > 0)
        {
            rb.linearVelocity = new Vector2(rb.linearVelocity.x, 0);
        }

        transform.position = new Vector2(transform.position.x, alturaMaxima);
    }
}
```

Ilustración 27: RanaMovement 2

Además de estas funciones, este script también tiene Revivir() pone a false la variable muerto y detiene el movimiento, Ganar() que busca el objeto MenuGanar en la jerarquía, pausa la música del nivel, activa la música de ganar la partida y muestra el menú de ganar mediante el objeto MenuGanar, y OnTriggerEnter2D() para que si el jugador colisiona con el objeto con el tag Bandera se lance la función Ganar().

```
private void Jump()
{
    audioSource.PlayOneShot(sonidoSaltar);
    rb.linearVelocity = new Vector2(rb.linearVelocity.x, JumpForce);
}

public void Revivir()
{
    muerto = false;
    rb.linearVelocity = Vector2.zero;
}

public void Ganar()
{
    MenuGanar menu = FindAnyObjectByType<MenuGanar>();

    if (menu != null && musicaFondo.isPlaying)
    {
        musicaFondo.Stop();
        audioSource2.PlayOneShot(sonidoGanar);
        menu.MostrarGanar();
    }
}

public void Morir()
{
    MenuMorir menu = FindAnyObjectByType<MenuMorir>();

    if (menu != null)
    {
        audioSource3.PlayOneShot(sonidoPerder);
        menu.MostrarGameOver();
    }
}

private void OnTriggerEnter2D(Collider2D collision)
{
    if (collision.CompareTag("Bandera"))
    {
        Ganar();
    }
}
```

Ilustración 28: RanaMovement 3

3.5. DatosJuego

El script 'DatosJuego.cs' contiene los datos que se van a guardar en los archivos .json cuando se guarde partida. Estos datos son: posicion del jugador, número de vidas, nombre de la partida, fecha, puntuación obtenida, tiempo jugado.

```
public class DatosJuego
{
    public Vector3 posicion;
    public int vida;
    public string nombrePartida;
    public string fecha;
    public float puntuacion;
    public float tiempoJugado;
}
```

Ilustración 29: DatosJuego

3.6. ControladorDatosJuego

Este script es el sistema de guardado y carga completo del juego. Maneja posición, vidas, puntos, tiempo y distintos slots de partidas. Utiliza un patrón Singleton mediante Instancia para que mediante la función Awake() al cambiar de escena el objeto viva entre escenas y mantiene datos globales. Tiene una referencia al jugador, a los puntos y al tiempo de la partida.

La función OnDestroy() limpia el evento para evitar errores de controladores duplicados al cambiar de escena.

Mediante Update() se tiene en cuenta la carga y el guardado de datos con atajos mediante teclado.

```
public class ControladorDatosJuego : MonoBehaviour
{
    private const int NUMERO_VIDAS = 6;
    public static ControladorDatosJuego Instancia;

    public GameObject jugador;
    public DatosJuego datosJuego = new DatosJuego();
    private Cronometro cronometro;
    private Puntuaje puntuaje;

    private void Awake()
    {
        if (Instancia != null)
        {
            Destroy(gameObject);
            return;
        }

        Instancia = this;
        DontDestroyOnLoad(gameObject);
        SceneManager.sceneLoaded += OnSceneLoaded;
    }

    private void OnDestroy()
    {
        SceneManager.sceneLoaded -= OnSceneLoaded;
    }

    private void Update()
    {
        if (Keyboard.current.cKey.wasPressedThisFrame) CargarDatos();
        if (Keyboard.current.gKey.wasPressedThisFrame) GuardarDatos();
    }
```

Ilustración 30: ControladorDatosJuego

La función OnSceneLoaded() reseta referencias y carga los datos.

ObtenerSlotActual() utiliza PlayerPrefs y devuelve el slot seleccionado, si no existe utiliza el slot 1.

ObtenerRutaArchivo() devuelve la ruta donde se encuentra el archivo JSON de la partida seleccionada.

La función ObtenerJugador() busca al jugador dinámicamente y guarda la referencia.

```
private void OnSceneLoaded(Scene scene, LoadSceneMode mode)
{
    jugador = null;
    puntuaje = FindAnyObjectByType<Puntuaje>();

    Debug.Log("Escena cargada — intentando cargar datos");
    CargarDatos();
}

int ObtenerSlotActual()
{
    return PlayerPrefs.GetInt("SlotSeleccionado", 1);
}

string ObtenerRutaArchivo()
{
    int slot = ObtenerSlotActual();
    return Path.Combine(Application.persistentDataPath, "partida_" + slot + ".json");
}

GameObject ObtenerJugador()
{
    if (jugador == null)
    {
        RanaMovement rana = FindAnyObjectByType<RanaMovement>(FindObjectsInactive.Include);
        if (rana != null)
        {
            jugador = rana.gameObject;
        }
    }
    return jugador;
}
```

Ilustración 31: ControladorDatosJuego 2

La función CargarDatos() es la encarga de traer el archivo JSON de la partida existente guardada al nivel. Lee el Json, lo que conlleva restaurar los datos del juego a los que tiene el archivo.

```
public void CargarDatos()
{
    string ruta = ObtenerRutaArchivo();
    if (!File.Exists(ruta))
    {
        Debug.Log("No hay archivo de guardado → empezando desde cero");
        return;
    }

    string contenido = File.ReadAllText(ruta);
    datosJuego = JsonUtility.FromJson<DatosJuego>(contenido);

    GameObject j = ObtenerJugador();
    if (j != null)
    {
        j.transform.position = datosJuego.posicion;

        VidaJugador vida = j.GetComponentInChildren<VidaJugador>(true);
        if (vida != null)
        {
            vida.cantidadVida = datosJuego.vida;
            vida.ActualizarCorazonesUI();
        }
    }

    if (puntuaje == null) puntuaje = FindAnyObjectByType<Puntuaje>();
    if (puntuaje != null) puntuaje.EstablecerPuntos(datosJuego.puntuacion);

    if (cronometro == null) cronometro = FindAnyObjectByType<Cronometro>();
    if (cronometro != null)
    {
        cronometro.EstablecerTiempo(datosJuego.tiempoJugado);
    }

    Debug.Log("CARGADO slot " + ObtenerSlotActual() +
              " | Puntos: " + datosJuego.puntuacion +
              " | Tiempo: " + datosJuego.tiempoJugado);
}
```

Ilustración 32: CargarDatos

La funcioón GuardarDatos() se encarga de obtener todos los datos necesarios que se guardan en DatosJuego de la partida existente, los convierte en JSON y los guarda.

```
public void GuardarDatos()
{
    string ruta = ObtenerRutaArchivo();

    GameObject j = ObtenerJugador();
    if (j == null) return;

    datosJuego.posicion = j.transform.position;
    datosJuego.nombrePartida = "Partida " + ObtenerSlotActual();
    datosJuego.fecha = DateTime.Now.ToString("dd/MM/yyyy HH:mm");

    VidaJugador vida = j.GetComponentInChildren<VidaJugador>(true);

    if (vida != null)
    {
        datosJuego.vida = vida.cantidadVida;
    }
    else
    {
        datosJuego.vida = NUMERO_VIDAS;
    }

    if (puntuaje == null) puntuaje = FindAnyObjectByType<Puntuaje>();
    if (puntuaje != null) datosJuego.puntuacion = puntuaje.ObtenerPuntos();
    else datosJuego.puntuacion = 0f;

    if (cronometro == null) cronometro = FindAnyObjectByType<Cronometro>();
    if (cronometro != null)
    {
        datosJuego.tiempoJugado = cronometro.tiempoActual;
    }
    else
    {
        datosJuego.tiempoJugado = 0f;
    }

    string json = JsonUtility.ToJson(datosJuego, true);
    File.WriteAllText(ruta, json);

    Debug.Log("PARTIDA GUARDADA - Vidas: " + datosJuego.vida);
}
```

Ilustración 33: GuardarDatos

Por último, la función BorrarPartidaActual() elimina el archivo del slot actual, está función se lanza cuando el jugador se pasa el nivel.

```
public void BorrarPartidaActual()
{
    string ruta = ObtenerRutaArchivo();

    if (File.Exists(ruta))
    {
        File.Delete(ruta);
        Debug.Log("Partida borrada correctamente");
    }
    else
    {
        Debug.Log("No había partida para borrar");
    }
}
```

Ilustración 34: BorrarPartidaActual

3.7. DatosRanking

Este archivo contiene la estructura de los datos que se van a guardar en los archivos JSON para mantener el ranking. El script lo forman la clase EntradaRanking que guarda el tiempo, puntuación vidas y fecha, y la clase DatosRankings que es un contenedor para poder guardar múltiples entradas al ranking en el JSON.

```
using System;
using System.Collections.Generic;

// Estructura para cada fila del ranking
[Serializable]
public class EntradaRanking
{
    public float tiempo;
    public float puntos;
    public int vidas;
    public string fecha;
}

// Clase contenedora para JSON
[Serializable]
public class DatosRankings
{
    public List<EntradaRanking> listaRankings = new List<EntradaRanking>();
}
```

Ilustración 35: DatosRanking

3.8. GestorRankings

Este script guarda los datos, los carga, los ordena y genera el texto para poder mostrarlo. Contiene una Instancia única, al ser un Singleton (static), que permite que se acceda desde cualquier script.

Mediante la función Awake() se logra que esta Instancia no se destruya en el cambio de escenas, para así poder mantener los datos de la partida cuando se cambie a la escena del menú inicial.

```
public class GestorRankings : MonoBehaviour
{
    public static GestorRankings Instancia;
    private string rutaArchivo;

    private void Awake()
    {
        if (Instancia == null)
        {
            Instancia = this;
            DontDestroyOnLoad(gameObject);
        }
        else
        {
            Destroy(gameObject);
            return;
        }
        rutaArchivo = Path.Combine(Application.persistentDataPath, "rankings.json");
    }
```

Ilustración 36: GestorRankings

La función CargarRankings() se encarga de leer el archivo JSON que contiene los rankings y los ordena por tiempo, puntuación y número de vidas en ese orden. En caso de empatar en tiempo el ranking se decidirá por puntuación, y en caso de empatar en tiempo y puntuación se decidirá por vidas.

GuardarNuevaPuntuacion() se encarga de guardar el nuevo recórd, primero carga los datos ya existentes en el ranking, crea una nueva entrada con nuevos datos, y se añade a la lista de forma ordenada. Por último, convierte los datos a JSON y lo guarda.

```
public void GuardarNuevaPuntuacion(float tiempo, float puntos, int vidas)
{
    DatosRankings datos = CargarRankings();
    EntradaRanking nuevaEntrada = new EntradaRanking
    {
        tiempo = tiempo,
        puntos = puntos,
        vidas = vidas,
        fecha = DateTime.Now.ToString("dd/MM/yyyy HH:mm")
    };
    datos.listaRankings.Add(nuevaEntrada);
    datos.listaRankings = datos.listaRankings.OrderBy(t => Mathf.FloorToInt(t.tiempo)).ThenByDescending(t => t.puntos).ThenByDescending(t => t.vidas).Take(10).ToList();

    File.WriteAllText(rutaArchivo, JsonUtility.ToJson(datos, true));
}

public DatosRankings CargarRankings()
{
    if (!File.Exists(rutaArchivo)) return new DatosRankings();

    var datos = JsonUtility.FromJson<DatosRankings>(File.ReadAllText(rutaArchivo));

    datos.listaRankings = datos.listaRankings
        .OrderBy(t => Mathf.FloorToInt(t.tiempo))
        .ThenByDescending(t => t.puntos)
        .ThenByDescending(t => t.vidas)
        .ToList();

    return datos;
}
```

Ilustración 37: GestorRankings 2

Como última función del script, ObtenerTextoFormateado(), si no hay datos muestra que aún no hay recórds registrados, en cualquier otro caso, recorre la lista del ranking, convierte el tiempo a minutos:segundos y devuelve la lista completa como texto.

```
public string ObtenerTextoFormateado()
{
    DatosRankings datos = CargarRankings();
    if (datos.listaRankings == null || datos.listaRankings.Count == 0)
        return "TOP 10 MEJORES TIEMPOS\n\nNo hay récords registrados.";

    string textoFinal = "TOP 10 MEJORES TIEMPOS\n\n";
    int puesto = 1;
    foreach (var entrada in datos.listaRankings)
    {
        int min = Mathf.FloorToInt(entrada.tiempo / 60);
        int seg = Mathf.FloorToInt(entrada.tiempo % 60);
        textoFinal += $"{puesto}. {min:00}:{seg:00} | Pts: {entrada.puntos} | Vidas: {entrada.vidas}\n";
        puesto++;
    }
    return textoFinal;
}
```

Ilustración 38: ObtenerTextoFormateado

3.9. VisualizadorRankingsUI

Este script muestra el ranking en pantalla mediante un variable TextMeshProUGUI. OnEnable() se ejecuta al activar la función y llama a su vez a ActualizarUI(), encargada de pedir los datos a la instancia del gestor, convertirlos en texto y mostrarlos.

```
using TMPro;
using UnityEngine;

public class VisualizadorRankingsUI : MonoBehaviour
{
    [SerializeField] private TextMeshProUGUI textoRankings;

    private void OnEnable()
    {
        ActualizarUI();
    }

    public void ActualizarUI()
    {
        if (GestorRankings.Instancia != null && textoRankings != null)
        {
            textoRankings.text = GestorRankings.Instancia.ObtenerTextoFormateado();
            Debug.Log("Ranking actualizado desde el Gestor.");
        }
    }
}
```

Ilustración 39: VisualizadorRankingsUI

3.9. VidaJugador

'VidaJugador.cs' controla la cantida de vidas, el daño de los enemigos y vulnerabilidad al ser golpeado. Como variables tiene NUMERO_VIDAS constante, definido como un máximo de 6 vidas, la variable cantidadVida que cambia durante el juego, la fuerza de rebote y de empuje lateral, el tiempo que está invulnerable el jugador. Define también un array de GameObject para los corazones del HUD, un Rigidbody2D para la física del jugador, RanaMovement para el control del jugador y el sonido del golpe con los enemigos.

Awake() inicializa la vida al máximo, Start() obtiene las referencias necesarias e inicializa los corazones mediante InicializarCorazones(), busca el objeto llamado VIDAS en la escena, por cada hijo que encuentre en la jerarquía es igual a un corazón y los guarda en el array corazones; y los actualiza mediante ActualizarCorazonesUI() activando tan solo los corazones en consonancia con las vidas que le quedan al jugador.

```
public class VidaJugador : MonoBehaviour
{
    private const int NUMERO_VIDAS = 6;
    public int cantidadVida;

    public float fuerzaRebote = 3f;
    public float fuerzaEmpujeLateral = 5f;

    public float tiempoInvulnerabilidad = 1f;
    public float tiempoPerdidaControl = 0.3f;

    public GameObject[] corazones;

    private Rigidbody2D rb;
    private RanaMovement movement;
    private bool puedeRecibirDano = true;

    [SerializeField] private AudioSource audioSource;
    [SerializeField] private AudioClip sonidoGolpe;

    void Awake()
    {
        cantidadVida = NUMERO_VIDAS;
    }

    void Start()
    {
        rb = GetComponent<Rigidbody2D>();
        movement = GetComponent<RanaMovement>();

        InicializarCorazones();
        ActualizarCorazonesUI();
    }
```

Ilustración 40: VidaJugador

```
private void InicializarCorazones()
{
    GameObject contenedorVidas = GameObject.Find("VIDAS");

    if (contenedorVidas != null)
    {
        corazones = new GameObject[contenedorVidas.transform.childCount];

        for (int i = 0; i < corazones.Length; i++)
        {
            corazones[i] = contenedorVidas.transform.GetChild(i).gameObject;
        }
    }
}

public void ActualizarCorazonesUI()
{
    if (corazones == null || corazones.Length == 0)
        InicializarCorazones();

    for (int i = 0; i < corazones.Length; i++)
    {
        corazones[i].SetActive(i < cantidadVida);
    }
}
```

Ilustración 41: VidaJugador 2

RecibirDanioDesdeAbeja() es la encarga de gestionar el encuentro jugador con abeja. Si el jugador está encima de la abeja no recibirá daño, en caso contrario el jugador recibe un empuje lateral y se llama a la función AplicarDanio() con 2 de daño. AplicarDanio() resta el número de vidas indicado en el parámetro de la función, emite el sonido y actualiza el HUD de los corazones. En caso de morir, llama al sistema de muerte del jugador, si sigue vivo activa invulnerabilidad mediante la función ActivarInvulnerabilidad() que evita el daño en cadena y le quita el control al jugador durante unos pocos segundos y se lo devuelve.

```
public void RecibirDanioDesdeAbeja(Vector2 posicionAbeja)
{
    if (!puedeRecibirDano) return;

    float diferenciaY = transform.position.y - posicionAbeja.y;

    //empuje
    if (diferenciaY > 0.8f)
    {
        float direccion = transform.position.x < posicionAbeja.x ? -1f : 1f;

        rb.linearVelocity = new Vector2(direccion * fuerzaEmpujeLateral, 2f);
        return;
    }

    AplicarDanio(2, posicionAbeja);
}

private void AplicarDanio(int dano, Vector2 posicionEnemigo)
{
    cantidadVida -= dano;

    audioSource?.PlayOneShot(sonidoGolpe);
    ActualizarCorazonesUI();

    Vector2 direccion = ((Vector2)transform.position - posicionEnemigo).normalized;

    rb.linearVelocity = new Vector2(
        direccion.x * fuerzaRebote,
        fuerzaRebote * 0.9f
    );

    if (cantidadVida <= 0)
    {
        movement?.Morir();
    }
    else
    {
        StartCoroutine(ActivarInvulnerabilidad());
    }
}
```

Ilustración 42: VidaJugador 3

Por último, la función OnCollisionEnter2D() es para cuando el jugador toca al enemigo normal (seta). La función busca al objeto con el tag Enemy y aplica el daño de una vida.

```
private IEnumerator ActivarInvulnerabilidad()
{
    puedeRecibirDano = false;

    if (movement != null)
        movement.enabled = false;

    yield return new WaitForSeconds(tiempoPerdidaControl);

    if (movement != null)
        movement.enabled = true;

    yield return new WaitForSeconds(tiempoInvulnerabilidad - tiempoPerdidaControl);

    puedeRecibirDano = true;
}

private void OnCollisionEnter2D(Collision2D collision)
{
    if (!puedeRecibirDano) return;

    if (collision.gameObject.CompareTag("Enemy"))
    {
        AplicarDanio(1, collision.transform.position);
    }
}
```

Ilustración 43: VidaJugador 4

3.10. SETA_PATROL

El script 'SETA_PATROL.cs' controla al enemigo (seta) que patrulla entre dos puntos. El funcionamiento es el siguiente: la seta se mueve entre dos puntos (A y B), cuando llega a uno de los dos puntos cambia de dirección hacia el otro, volteándose el sprite y manteniendo la animación. Tiene dos variables GameObject para los puntos, un Rigidbody2D para el movimiento, Animator para la animación, un Transform currentPoint para saber hacia dónde se dirige y un float speed para controlar la velocidad de la seta.

La función Start() obtiene los componentes y empieza moviéndose hacia el punto B, activando la animación de caminar mediante el booleano isRunning configurando en el Animator.

```
public class SETA_PATROL : MonoBehaviour
{
    public GameObject pointA;
    public GameObject pointB;
    private Rigidbody2D rb;
    private Animator anim;
    private Transform currentPoint;
    public float speed;

    void Start()
    {
        rb = GetComponent<Rigidbody2D>();
        anim = GetComponent<Animator>();
        currentPoint = pointB.transform;
        anim.SetBool("isRunning", true);
    }
```

Ilustración 44: SETA_PATROL

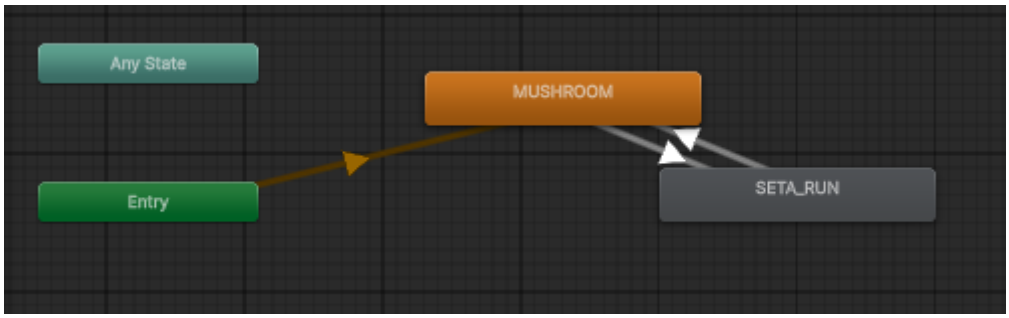

Ilustración 45: AnimatorSeta

Update() para el movimiento, si va hacia B se mueve a la derecha y si va a A se mueve a la izquierda. En el caso de esté cerca del punto, llama a la función flip() para girar el sprite mediante la dirección horizontal y, cambia el punto al que ir ahora.

Por último, OnDrawGizmos() dibuja en el editor una pequeña esfera en cada punto y una línea entre ambos.

Ilustración 46: PuntosSeta

```
void Update()
{
    Vector2 point = currentPoint.position - transform.position;

    if (currentPoint == pointB.transform)
    {
        rb.linearVelocity = new Vector2(speed, 0);
    }
    else
    {
        rb.linearVelocity = new Vector2(-speed, 0);
    }

    if (Vector2.Distance(transform.position, currentPoint.position) < 0.5f && currentPoint == pointB.transform)
    {
        flip();
        currentPoint = pointA.transform;
    }

    if (Vector2.Distance(transform.position, currentPoint.position) < 0.5f && currentPoint == pointA.transform)
    {
        flip();
        currentPoint = pointB.transform;
    }
}

private void flip()
{
    Vector3 localScale = transform.localScale;
    localScale.x *= -1;
    transform.localScale = localScale;
}

private void OnDrawGizmos()
{
    if (pointA != null && pointB != null)
    {
        Gizmos.DrawWireSphere(pointA.transform.position, 0.5f);
        Gizmos.DrawWireSphere(pointB.transform.position, 0.5f);
        Gizmos.DrawLine(pointA.transform.position, pointB.transform.position);
    }
}
```

Ilustración 47: SETA_PATROL 2

3.11. ABEJA_PATROL

El script 'ABEJA_PATROL.cs' maneja la técnica de IA máquina de estados junto a un movimiento con evasión de objetos y un comportamiento de decisión del enemigo frente al jugador.

Más adelante en el punto "5. Técnica Inteligencia Artificial" se detalla el comportamiento del enemigo abeja con respecto al jugador, de momento vamos a ver que contiene el script:

Variables velocidad y suavizado de movimiento, conjunto de puntos entre los que la abeja patrulla y se mueve, objeto Transform jugador, los distintos radios en los que puede cambiar la actitud de la abeja según la cercanía del jugador (histéresis), el tiempo de cooldown, la distancia y tiempo de retirada, la distancia y la fuerza de detección y evasión del terreno y, el tiempo de decisión de camino (izquierda o derecha).

```
[Header("Movimiento")]
[SerializeField] private float velocidadMovimiento = 1.5f;
[SerializeField] private float suavizado = 3f;
private Vector2 velocidadActual;

[Header("Patrulla")]
[SerializeField] private Transform[] puntosMovimiento;
[SerializeField] private float distanciaMinima = 0.2f;
private int indicePatrulla = 0;
private int direccionPatrulla = 1;

[Header("Jugador")]
[SerializeField] private Transform jugador;
[SerializeField] private float distanciaMinimaJugador = 1.2f;

[Header("Radios")]
[SerializeField] private float radioPersecucionEntrada = 6f;
[SerializeField] private float radioPersecucionSalida = 8f;
[SerializeField] private float radioAtaqueEntrada = 2f;
[SerializeField] private float radioAtaqueSalida = 3f;

[Header("Ataque")]
[SerializeField] private float cooldownAtaque = 1.5f;
private float tiempoUltimoAtaque;

[Header("Retirada")]
[SerializeField] private float distanciaRetirada = 3f;
[SerializeField] private float tiempoRetirada = 1.2f;
private float timerRetirada;

[Header("Evitar obstáculos")]
[SerializeField] private LayerMask capaTerreno;
[SerializeField] private float distanciaDeteccion = 2.5f;
[SerializeField] private float fuerzaEvasion = 2f;

[Header("Decisión")]
[SerializeField] private float tiempoDecision = 0.3f;
private float timerDecision;
private Vector2 direccionGuardada;

[Header("Referencias")]
[SerializeField] private Animator animator;

private Rigidbody2D rb;
private SpriteRenderer spriteRenderer;

private EstadoAbeja estadoActual;
```

Ilustración 48: Variables ABEJA_PATROL

En Start() se obtienen los componentes y empieza en estado de patrulla.

```
void Start()
{
    rb = GetComponent<Rigidbody2D>();
    spriteRenderer = GetComponent<SpriteRenderer>();

    if (animator == null)
        animator = GetComponent<Animator>();

    if (jugador == null)
    {
        GameObject obj = GameObject.FindGameObjectWithTag("Player");
        if (obj != null) jugador = obj.transform;
    }

    CambiarEstado(EstadoAbeja.Patrullando);
}
```

Ilustración 49: Start ABEJA_PATROL

En FixedUpdate() se encuentra el comportamiento de la máquina de estados que se explicará más adelante. Hace uso de las funciones Patrullar(), Perseguir(), CambiarEstado(), Atacar() y Retirarse().

OnTriggerStay2D() llama a la función RecibirDanioDesdeAbeja del objeto VidaJugador al colisionar la abeja con el jugador.

MoverConEvasion() se encarga de detectar obstáculos mediante RaycastHit2D, si hay obstáculos obtiene la dirección de la superficie y calcula dos tangentes (direcciones paralelas a la pared) y elige la mejor.

Girar() hace que el enemigo mire hacia el jugador o el destino.

```
void MoverConEvasion(Vector2 destino)
{
    Vector2 direccion = (destino - rb.position).normalized;

    RaycastHit2D hit = Physics2D.Raycast(rb.position, direccion, distanciaDeteccion, capaTerreno);

    if (hit.collider != null)
    {
        Vector2 normal = hit.normal;

        Vector2 tangente1 = new Vector2(-normal.y, normal.x);
        Vector2 tangente2 = new Vector2(normal.y, -normal.x);

        float dot1 = Vector2.Dot(tangente1, direccion);
        float dot2 = Vector2.Dot(tangente2, direccion);

        Vector2 direccionEvasion = (dot1 > dot2) ? tangente1 : tangente2;

        direccion = (direccionEvasion * fuerzaEvasion + normal).normalized;
    }

    velocidadActual = Vector2.Lerp(
        velocidadActual,
        direccion * velocidadMovimiento,
        suavizado * Time.fixedDeltaTime
    );

    rb.MovePosition(rb.position + velocidadActual * Time.fixedDeltaTime);
}

void Girar(float objetivoX)
{
    if (rb.position.x < objetivoX)
        spriteRenderer.flipX = false;
    else if (rb.position.x > objetivoX)
        spriteRenderer.flipX = true;
}
```

Ilustración 50: ABEJA_PATROL

OnDrawGizmosSelected() muestra los radios por colores en el editor.

```
// RADIOS
void OnDrawGizmosSelected()
{
    Gizmos.color = Color.yellow;
    Gizmos.DrawWireSphere(transform.position, radioPersecucionEntrada);

    Gizmos.color = Color.cyan;
    Gizmos.DrawWireSphere(transform.position, radioPersecucionSalida);

    Gizmos.color = Color.red;
    Gizmos.DrawWireSphere(transform.position, radioAtaqueEntrada);

    Gizmos.color = Color.magenta;
    Gizmos.DrawWireSphere(transform.position, radioAtaqueSalida);
}
```

Ilustración 51: OnDrawGizmosSelected

3.12. Moneda

El script “Moneda.cs” contiene la clase Moneda. Esta a su vez tiene los siguientes atributos:

```
private Puntuaje puntos;
public static List<Moneda> todasLasMonedas = new List<Moneda>();

[SerializeField] private int cantidadPuntos;
[SerializeField] private AudioSource audioSource1;
[SerializeField] private AudioSource audioSource2;
[SerializeField] private AudioClip sonidoMoneda;
[SerializeField] private AudioClip sonidoMonedaGrande;
```

Ilustración 52. Moneda 1.

Como podemos observar hay un array estático con todas las Monedas que tendremos en el juego, la cantidad de puntos que da la moneda al ser activada por el jugador y los dos tipos de sonidos para los dos tipos de monedas que habrá en el juego.

```
void Awake()
{
    todasLasMonedas.Add(this);
}

void OnDestroy()
{
    todasLasMonedas.Remove(this);
}

void OnEnable()
{
    if (puntos == null)
        puntos = FindAnyObjectByType<Puntuaje>();
}
```

Ilustración 53. Moneda 2.

El método Awake() se llama cada vez que una instancia de moneda se crea. Este añade la moneda al array estático. La función OnDestroy() elimina la moneda del array una vez el objeto se destruye. La función OnEnable() hace que se asigne la referencia al objeto puntuaje cada vez que la moneda se activa.

```
private void OnTriggerEnter2D(Collider2D other)
{
    if (other.CompareTag("Player"))
    {
        puntos?.SumarPuntos(cantidadPuntos);

        if (cantidadPuntos > 50)
        {
            audioSource2.PlayOneShot(sonidoMonedaGrande);
        }
        else
        {
            audioSource1.PlayOneShot(sonidoMoneda);
        }

        gameObject.SetActive(false);
    }
}
```

Ilustración 54. Moneda 3

Por último, el método OnTriggerEnter2D(Coollider2D other) se encarga de comprobar si un objeto con la etiqueta “Player” colisiona con la moneda. Si está colisionando se le suman el número de puntos correspondiente a cada moneda. Si puntos es NULL el operador ‘?’ se encargaría de que la línea de código no se ejecutase. Por último, se reproduciría un sonido teniendo en cuenta si es una moneda más o menos valiosa y se ocultaría el objeto.

3.13. Trampoline

El script ‘Trampoline.cs’ contiene la clase Trampolín, con los siguientes atributos:

```
public class Trampolin : MonoBehaviour
{
    public float fuerzaSalto = 18f;
    public Animator animator;
    [SerializeField] private AudioSource audioSource;
    [SerializeField] private AudioClip sonidoTrampolin;

    private float tiempoCooldown = 0.2f;
    private float tiempoUltimoSalto = 0f;
```

Ilustración 55.Trampolín 1

Podemos ver la fuerza de Salto, la animación del Sprite, la fuente de audio junto al sonido, el temporizador para poder volver a saltar en el trampolín y el tiempo del último salto.

El método de esta clase es OnCollisionEnter2D(Collision2D collision).

Esta detecta si el objeto con etiqueta “Player” está colisionando con el trampolín, solo saltará si ha pasado suficiente tiempo desde el último salto y si está con una velocidad menor a 0.05f.

```
private void OnCollisionEnter2D(Collision2D collision)
{
    if (collision.gameObject.CompareTag("Player"))
    {
        // Comprobamos si ha pasado suficiente tiempo desde el último salto
        if (Time.time >= tiempoUltimoSalto + tiempoCooldown)
        {
            Rigidbody2D rb = collision.gameObject.GetComponent<Rigidbody2D>();

            // Solo si cae hacia abajo (o está en el punto exacto de 0)
            if (rb != null && rb.linearVelocity.y <= 0.05f)
            {
                tiempoUltimoSalto = Time.time;
                audioSource.PlayOneShot(sonidoTrampolin);

                animator.SetTrigger("Bounce");

                rb.linearVelocity = new Vector2(rb.linearVelocity.x, 0f);
                rb.AddForce(Vector2.up * fuerzaSalto, ForceMode2D.Impulse);
            }
        }
    }
```

Ilustración 56.Trampolín 2

3.14. Teletransporte

El script ‘Teletransporte.cs’ se encarga de llevar al jugador de una posición a otra cuando este colisiona con el interactuable. Podemos ver que los únicos atributos son el canal de audio junto al sonido y el punto a donde se va a llevar al personaje.

El método Start() se invoca una sola vez, si el puntoDestino es nulo y hay hijos en la jerarquía, se asignará la posición del hijo a puntoDestino.

La función es OnTriggerEnter2D(Collider2D collision) hace que la posición del jugador sea la que tenemos como atributo.

```
public class Teletransporte : MonoBehaviour
{
    [Header("Destino")]
    [SerializeField] private Transform puntoDestino;
    [SerializeField] private AudioSource audioSource;
    [SerializeField] private AudioClip sonidoTeleport;
    void Start()
    {
        if (puntoDestino == null && transform.childCount > 0)
        {
            puntoDestino = transform.GetChild(0);
        }
    }
    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (collision.CompareTag("Player"))
        {
            Rigidbody2D rb = collision.GetComponent<Rigidbody2D>();

            if (rb != null)
            {
                audioSource.PlayOneShot(sonidoTeleport);
                rb.position = puntoDestino.position;
                rb.linearVelocity = Vector2.zero;
            }
        }
    }
}
```

Ilustración 57. Teletransporte

3.15. Puntuaje

El script 'Puntuaje.cs' contiene la clase Puntuaje, esta tiene los siguientes atributos:

```
public class Puntuaje : MonoBehaviour
{
    private float puntos;
    private TextMeshProUGUI textMesh;
```

Ilustración 58.Puntuaje 1

El método awake() obtiene el componente textmesh al instanciar el objeto, Start() simplemente llama a la función actualizarTexto(). SumarPuntos(float puntosM) añade los puntos pasados por parámetro al atributo de la clase y ObtenerPuntos() devuelve el atributo puntos.

El método EstablecerPuntos(float NuevosPuntos) sobrescribe el contenido del atributo puntos.

Por último, el método ActualizarTexto() actualiza el componente (texto en pantalla) con los puntos actuales.

```
void Awake()
{
    textMesh = GetComponent<TextMeshProUGUI>();
}

void Start()
{
    ActualizarTexto();
}

public void SumarPuntos(float puntosM)
{
    puntos += puntosM;
    ActualizarTexto();
}

public float ObtenerPuntos()
{
    return puntos;
}
```

Ilustración 59. Puntuaje 2

```
public void EstablecerPuntos(float nuevosPuntos)
{
    puntos = nuevosPuntos;
    ActualizarTexto();
}

private void ActualizarTexto()
{
    if (textMesh != null)
    {
        textMesh.text = "Puntos: " + puntos;
    }
}
```

Ilustración 60. Puntuaje 3

3.16. Cronometro

El script 'Cronometro.cs' contiene la clase Cronometro. Esta clase se encarga de llevar el tiempo exacto que el usuario está jugando. Tiene el método PausarTiempo() utilizado en los menús de pausa y morir, ReanudarTiempo()

también usada en estos menús una vez sales. EstablecerTiempo() se usa para cargar el tiempo de una partida existente en cualquiera de los tres slots.

La función ActualizarUI() muestra en la pantalla de juego el tiempo que llevamos en esa partida.

```
public class Cronometro : MonoBehaviour
{
    public float tiempoActual = 0f;
    private bool cronometroActivo = true;

    [SerializeField] private TextMeshProUGUI textoTiempo;

    void Update()
    {
        if (cronometroActivo)
        {
            tiempoActual += Time.deltaTime * Time.timeScale;
            ActualizarUI();
        }
    }

    public void PausarTiempo() { cronometroActivo = false; }
    public void ReanudarTiempo() { cronometroActivo = true; }
    public void EstablecerTiempo(float tiempoCargado) { tiempoActual = tiempoCargado; }

    private void ActualizarUI()
    {
        if (textoTiempo != null)
        {
            int minutos = Mathf.FloorToInt(tiempoActual / 60);
            int segundos = Mathf.FloorToInt(tiempoActual % 60);
            textoTiempo.text = string.Format("{0:00}:{1:00}", minutos, segundos);
        }
    }
}
```

Ilustración 61. Cronómetro

3.17. MENUINICIAL

El script 'MENUINICIAL.cs' es el encargado de realizar las funciones de los botones 'JUGAR' 'SALIR' y 'SONIDO'.

En este se encuentran el audiomixer y los objetos slider para manejar la música y efectos de sonido.

```
public class MENUINICIAL : MonoBehaviour
{
    [SerializeField] private AudioMixer audioMixer;
    [SerializeField] private Slider sliderVolumenMusica;
    [SerializeField] private Slider sliderVolumenSFX;
    private GameObject panelRankings;
```

Ilustración 62.MenuInicial 1

El método Start() se encargará de asignar el volumen guardado en el archivo de preferencias del jugador, si no existe se asignará como 1f;

```
void Start()
{
    float valorM = PlayerPrefs.GetFloat("VOLUMEN", 1f);
    float valorE = PlayerPrefs.GetFloat("SFX", 1f);

    sliderVolumenMusica.value = valorM;
    sliderVolumenSFX.value = valorE;

    CambiarVolumenMusica(valorM);
    CambiarVolumenSFX(valorE);
}
```

Ilustración 63. MenuInicial 2

El método Jugar() cambiará a la escena donde se elige la partida a cargar, Salir() cerrará la aplicación y CambiarVoluenMusica(float valor) junto a CambiarVolmenMusica(float valor) nos permitirán cambiar el volumen de los efectos de sonido y música tanto del menú principal como del juego.

```
public void Jugar()
{
    SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex + 1);
}

public void Salir()
{
    Debug.Log("Saliendo del juego...");
    Application.Quit();
    #if UNITY_EDITOR
    UnityEditor.EditorApplication.isPlaying = false;
    #endif
}

public void CambiarVolumenMusica(float valor)
{
    float db = valor > 0.0001f ? Mathf.Log10(valor) * 20 : -80f;
    audioMixer.SetFloat("VOLUMEN", db);
    PlayerPrefs.SetFloat("VOLUMEN", valor);
}

public void CambiarVolumenSFX(float valor)
{
    float db = valor > 0.0001f ? Mathf.Log10(valor) * 20 : -80f;
    audioMixer.SetFloat("SFX", db);
    PlayerPrefs.SetFloat("SFX", valor);
}
```

Ilustración 64.MenuInicial 3

3.18. CARGAR_PARTIDA

El script ‘CARGAR_PARTIDA.cs’ nos permitirá modificar el texto de los botones en el menú de cargar partida, mostrando una fecha o indicando que el slot no se ha seleccionado previamente.

El método SeleccionarSlot(int slot) guardará en el archivo de preferencias el slot seleccionado para posteriormente guardar los datos en el sitio correcto dentro del juego.

```
public TextMeshProUGUI textoInicio;
public Button[] botonesPartida;

void Start()
{
    ConfigurarMenu();
}

void ConfigurarMenu()
{
    string rutaBase = Application.persistentDataPath;

    for (int i = 0; i < botonesPartida.Length; i++)
    {
        int numeroPartida = i + 1;
        string rutaCompleta = Path.Combine(rutaBase, "partida_" + numeroPartida + ".json");

        TextMeshProUGUI textoBoton = botonesPartida[i].GetComponentInChildren<TextMeshProUGUI>();
        botonesPartida[i].onClick.RemoveAllListeners();

        if (File.Exists(rutaCompleta))
        {
            string contenido = File.ReadAllText(rutaCompleta);
            DatosJuego datos = JsonUtility.FromJson<DatosJuego>(contenido);
            textoBoton.text = "Partida " + numeroPartida + " - " + datos.fecha;
        }
        else
        {
            textoBoton.text = "Nueva Partida " + numeroPartida;
        }

        botonesPartida[i].onClick.AddListener(() => SeleccionarSlot(numeroPartida));
    }
}

void SeleccionarSlot(int slot)
{
    PlayerPrefs.SetInt("SlotSeleccionado", slot);
    SceneManager.LoadScene("proyecto");
}
```

Ilustración 65.CARGAR_PARTIDA

3.19. MenuGanar

El script ‘MenuGanar.cs’ contiene la clase MenuGanar, esta tiene los atributos cronometro y puntuaje inicializadas en el método Start() ejecutado una única vez.

Los otros dos atributos son el panel y el texto mostrado en el panel con la información de la partida.

```
public class MenuGanar : MonoBehaviour
{
    [SerializeField] private GameObject panelGanar;
    private TextMeshProUGUI texto;

    private Puntuaje puntuacion;
    private Cronometro cronometro;

    private void Start()
    {
        puntuacion = FindAnyObjectByType<Puntuaje>();
        cronometro = FindAnyObjectByType<Cronometro>();
    }
```

Ilustración 66.MenuGanar 1

El método MostrarGanar() muestra la puntuación junto al tiempo del cronometro y las vidas finales en el panelGanar. También guarda los datos en el archivo de rankings usando la función GuardarNuevaPuntuación() de GestorRankings. Por último, borra los datos del archivo correspondiente a la partida que ha acabado.

```
public void MostrarGanar()
{
    float tiempoFinal = 0f;
    float puntosFinales = 0f;
    int vidasFinales = 0;
    string textoTiempo = "00:00";

    if (cronometro != null)
    {
        cronometro.PausarTiempo();
        tiempoFinal = cronometro.tiempoActual;
        int minutos = Mathf.FloorToInt(tiempoFinal / 60);
        int segundos = Mathf.FloorToInt(tiempoFinal % 60);
        textoTiempo = string.Format("{0:00}:{1:00}", minutos, segundos);
    }

    if (puntuacion != null) puntosFinales = puntuacion.ObtenerPuntos();

    VidaJugador scriptVida = FindAnyObjectByType<VidaJugador>();
    if (scriptVida != null) vidasFinales = scriptVida.cantidadVida;

    panelGanar.SetActive(true);
    texto = panelGanar.GetComponentInChildren<TextMeshProUGUI>(true);

    if (texto != null)
    {
        texto.text = "¡Nivel Completado!\n" +
                     "Puntos: " + puntosFinales + "\n" +
                     "Tiempo: " + textoTiempo + "\n" +
                     "Vidas: " + vidasFinales;
    }

    try
    {
        if (GestorRankings.Instancia != null)
        {
            GestorRankings.Instancia.GuardarNuevaPuntuacion(tiempoFinal, puntosFinales, vidasFinales);
            Debug.Log("Ranking guardado correctamente.");
        }
    }
    catch (Exception e) { Debug.LogError("Error en Ranking: " + e.Message); }

    try
    {
        if (ControladorDatosJuego.Instancia != null)
        {
            ControladorDatosJuego.Instancia.BorrarPartidaActual();
            Debug.Log("Partida borrada tras guardar el ranking.");
        }
    }
    catch (Exception e) { Debug.LogError("Error al borrar: " + e.Message); }

    Time.timeScale = 0f;
}
```

Ilustración 67. MenuGanar 2

La función VolverMenu() cambia a la escena del menú principal asociada al botón mostrado en el panel y nos aseguramos de que el tiempo tenga velocidad normal.

```
public void VolverMenu()
{
    Time.timeScale = 1f;
    SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex - 1);
}
```

Ilustración 68. MenuGanar 3

3.20. MenuMorir

Los atributos de la clase menuMorir ('menuMorir.cs') son similares a los de menuGanar. También el mismo método Start().

```
public class MenuMorir : MonoBehaviour
{
    [SerializeField] private GameObject panelGameOver;
    private TextMeshProUGUI texto;

    private Puntuaje puntuacion;
    private Cronometro cronometro;

    private void Start()
    {
        puntuacion = FindAnyObjectByType<Puntuaje>();
        cronometro = FindAnyObjectByType<Cronometro>();
    }
```

Ilustración 69. MenuMorir 1

El método MostrarGameOver() hace visible el objeto panelGameOver, dentro de este muestra los puntos que obtuvimos, posteriormente pausa el tiempo.

Los métodos Reintentar() y Salir() reanudan el tiempo a velocidad normal y posteriormente cambian a la escena correspondiente.

```
public void MostrarGameOver()
{
    panelGameOver.SetActive(true);
    texto = panelGameOver.GetComponentInChildren<TextMeshProUGUI>(true);

    if (cronometro != null)
    {
        cronometro.PausarTiempo();
    }

    string textoPuntos = puntuacion != null ? puntuacion.ObtenerPuntos().ToString() : "0";

    if (texto != null)
    {
        texto.text = "Game Over\n" + "Puntos: " + textoPuntos;
    }

    Time.timeScale = 0f;
}

public void Reintentar()
{
    Time.timeScale = 1f;

    SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex);
}

public void Salir()
{
    Time.timeScale = 1f;
    SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex - 1);
}
```

Ilustración 70. MenuMorir 2

3.21. MenuPausa

En el script 'MenuPausa.cs' se encuentra la clase MenuPausa. Sus atributos son el objeto del botón pausa, el panel MPausa, el array de objetos tipo moneda y el cronómetro. En el método Start() se inicializan las monedas y el cronómetro.

```
public class MenuPausa : MonoBehaviour
{
    [SerializeField] private GameObject BotonPausa;
    [SerializeField] private GameObject MPausa;
    private GameObject[] monedas;
    private Cronometro cronometro;

    void Start()
    {
        monedas = GameObject.FindGameObjectsWithTag("Moneda");
        cronometro = FindAnyObjectByType<Cronometro>();
    }
```

Ilustración 71. MenuPausa 1

El método Pausa() hace que la velocidad del tiempo de juego se congele, también el cronómetro, oculta las monedas y las vidas mostradas por pantalla y muestra el menú de pausa. El método Reanudar() hace todo lo contrario.

```
public void Pausa()
{
    Time.timeScale = 0f;
    BotonPausa.SetActive(false);
    MPausa.SetActive(true);
    ActivarMonedas(false);
    ActivarVidas(false);
    if (cronometro != null)
    {
        cronometro.PausarTiempo();
    }
}

public void Reanudar()
{
    Time.timeScale = 1f;
    BotonPausa.SetActive(true);
    MPausa.SetActive(false);
    ActivarMonedas(true);
    ActivarVidas(true);
    if (cronometro != null)
    {
        cronometro.ReanudarTiempo();
    }
}
```

Ilustración 72. MenuPausa 2

El método Update() comprueba continuamente si se pulsa la tecla escape para ejecutar los métodos Reanudar() o Pausa() previamente explicados.

El método Guardar() invoca el método GuardarDatos() del ControladorDatosJuego que guarda en el archivo .json correspondiente a su slot los datos de la partida.

```
void Update()
{
    if (Keyboard.current.escapeKey.wasPressedThisFrame && !RanaMovement.Instancia.muerto)
    {
        if (Time.timeScale == 0f)
        {
            Reanudar();
        }
        else
        {
            Pausa();
        }
    }
}

public void Guardar()
{
    ControladorDatosJuego controlador = FindAnyObjectByType<ControladorDatosJuego>();

    if (controlador != null)
    {
        controlador.GuardarDatos();
        Debug.Log("Partida guardada desde menú pausa");
    }
    else
    {
        Debug.LogWarning("No se encontró ControladorDatosJuego");
    }
    Reanudar();
}
```

Ilustración 73. MenuPausa 2

Los métodos ActivarMonedas() y ActivarVidas() hacen que séan visibles los sprites dibujados y que se ocultan al pausar el juego. El método Salir() simplemente reanuda la velocidad normal del juego y cambia a la escena del menú principal.

```
public void Salir()
{
    Time.timeScale = 1f;
    SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex - 1);
}

void ActivarMonedas(bool estado)
{
    GameObject[] monedas = GameObject.FindGameObjectsWithTag("Moneda");

    foreach (GameObject moneda in monedas)
    {
        SpriteRenderer[] renderers = moneda.GetComponentsInChildren<SpriteRenderer>();

        foreach (SpriteRenderer sr in renderers)
        {
            sr.enabled = estado;
        }
    }
}

void ActivarVidas(bool estado)
{
    GameObject[] vidas = GameObject.FindGameObjectsWithTag("Vidas");

    foreach (GameObject vida in vidas)
    {
        SpriteRenderer[] renderers = vida.GetComponentsInChildren<SpriteRenderer>();
        foreach (SpriteRenderer sr in renderers)
        {
            sr.enabled = estado;
        }

        UnityEngine.UI.Image[] imagenes = vida.GetComponentsInChildren<UnityEngine.UI.Image>();
        foreach (UnityEngine.UI.Image img in imagenes)
        {
            img.enabled = estado;
        }
    }
}
```

Ilustración 74. MenuPausa 3

4. Persistencia de Datos y Preferencias

El jugador podrá guardar partida en cualquier momento, para ello deberá pulsar el botón guardar dentro del menú de pausa. Los datos que deberán ser guardados al cerrar el juego se encuentran en el script 'DatosJuego.cs'.

Se guardarán en un archivo .json (partida_1.json, partida_2.json o partida_3.json) dependiendo del slot elegido en el menú jugar.

El script 'ControladorDatosJuego.cs' realizará los guardados y cargas correspondientes a estos archivos.

También se utilizará el script 'GestorRanking.cs' para guardar y cargar en el menú de rankings los 10 primeros rankings. Los datos que se guardarán para el ranking se encuentran en el script 'DatosRanking.cs':

5. Técnica de inteligencia artificial

La técnica de IA utilizada es una máquina de estados en el enemigo abeja que tiene los siguientes estados: Patrullando, Persiguiendo, Atacando, Retirandose.

```
public enum EstadoAbeja
{
    Patrullando,
    Persiguiendo,
    Atacando,
    Retirandose
}
```

Ilustración 75. Enum EstadoAbeja

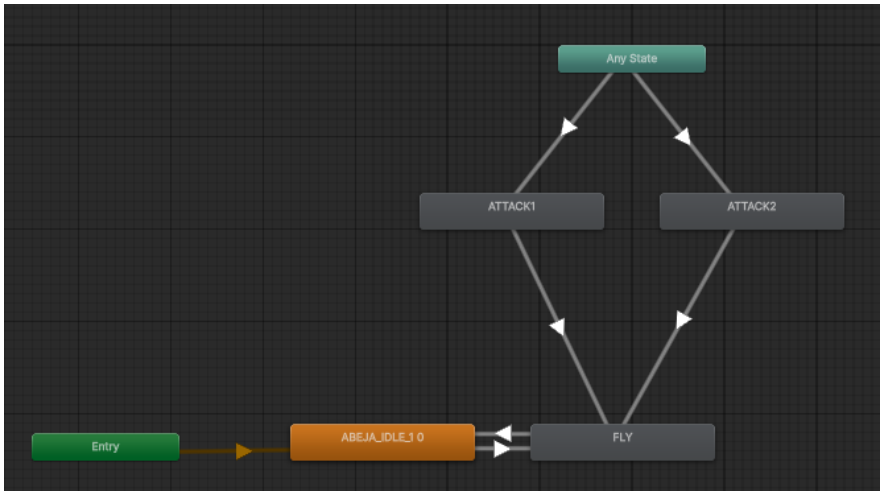

Ilustración 76. Diagrama de estados

Dentro del Animator del objeto se pueden ver los estados ABEJA_IDLE_0, FLY, ATTACK1 y ATTACK2, con las siguientes transiciones:

- ABEJA_IDLE_0 a FLY: está transición pone la variable isMoving a true y la animación de la abeja cambia para estar enfadada. Ocurre cuando el jugador está a una determinada distancia del enemigo y este comienza a perseguir al jugador.
- FLY a ABEJA_IDLE_0: ocurre cuando el jugador consigue alejarse de la abeja, poniendo la variable isMoving a false y cambiando la animación de enfadada a idle para seguir patrullando.
- ANY STATE a ATTACK1/ATTACK2: esta transición significa que da igual en el estado en el que esté la abeja puede atacar, para ello se tiene dos triggers attack1 que va a ATTACK1 y attack2 que va a ATTACK2, logrando un ataque inmediato. El tipo de ataque es aleatorio, hay un 50% de que realice uno de los ataques.
- ATTACK1/ATTACK2 a FLY: ocurre después de atacar, una vez ataque vuelve a estar enfadada. No tiene condiciones y debe tener Exit Time activado.

Con respecto al script, la máquina de estados se realiza sobre la función FixedUpdate().

```
void FixedUpdate()
{
    if (jugador == null)
    {
        Patrullar();
        return;
    }

    float distancia = Vector2.Distance(rb.position, jugador.position);
    bool jugadorPorEncima = jugador.position.y > rb.position.y + 0.5f;

    switch (estadoActual)
    {
        case EstadoAbeja.Patrullando:

            Patrullar();

            if (distancia < radioPersecucionEntrada)
                CambiarEstado(EstadoAbeja.Persiguiendo);

            break;

        case EstadoAbeja.Persiguiendo:

            Perseguir();

            if (distancia < radioAtaqueEntrada && !jugadorPorEncima)
                CambiarEstado(EstadoAbeja.Atacando);

            else if (distancia > radioPersecucionSalida)
                CambiarEstado(EstadoAbeja.Patrullando);

            break;
```

Ilustración 77. Estados Abeja 1

```
        case EstadoAbeja.Atacando:

            Perseguir();

            if (Time.time > tiempoUltimoAtaque + cooldownAtaque && !jugadorPorEncima)
            {
                Atacar();
            }

            if (distancia > radioAtaqueSalida || jugadorPorEncima)
                CambiarEstado(EstadoAbeja.Persiguiendo);

            break;

        case EstadoAbeja.Retirandose:

            Retirarse();

            break;
    }
}
```

Ilustración 78. Estados Abeja 2

Si el estado es Patrullando, se ejecuta Patrullar(): la abeja va hacia un punto, cuando llega va al siguiente, cuando logra ir a todos los puntos, hace efecto ping pong, es decir, no va al primer punto de nuevo, si no al anterior que visitó. Si la

distancia del jugador respecto a la abeja está dentro del radio de persecución, se cambia el estado a Persiguiendo.

```
void Patrullar()
{
    if (puntosMovimiento.Length == 0) return;

    Vector2 destino = puntosMovimiento[indicePatrulla].position;

    MoverConEvasion(destino);

    if (Vector2.Distance(rb.position, destino) < distanciaMinima)
    {
        indicePatrulla += direccionPatrulla;

        if (indicePatrulla >= puntosMovimiento.Length)
        {
            indicePatrulla = puntosMovimiento.Length - 2;
            direccionPatrulla = -1;
        }
        else if (indicePatrulla < 0)
        {
            indicePatrulla = 1;
            direccionPatrulla = 1;
        }
    }

    Girar(destino.x);
}
```

Ilustración 79. Patrullar

Si el estado es Persiguiendo, se ejecuta Perseguir(): recalcula la dirección y hace uso de MoverConEvasion() y Girar() para seguir al jugador en todo momento. Si la distancia respecto al jugador es menor al radio de ataque y el jugador no está encima de la abeja, el estado cambia a Atacando, en caso de que la distancia sea mayor que el radio de persecución, cambia al estado patrullando de nuevo.

```
void Perseguir()
{
    timerDecision -= Time.fixedDeltaTime;

    if (timerDecision <= 0)
    {
        Vector2 dir = jugador.position - transform.position;

        if (Mathf.Abs(dir.y) > 2f)
            dir.y *= 0.5f;

        direccionGuardada = dir.normalized;
        timerDecision = tiempoDecision;
    }

    if (Vector2.Distance(rb.position, jugador.position) > distanciaMinimaJugador)
    {
        MoverConEvasion(rb.position + direccionGuardada);
    }

    Girar(jugador.position.x);
}
```

Ilustración 80. Perseguir

Si el estado es Atacando, si el tiempo es superior al ultimo ataque más el cooldown de ataque se ejecuta Atacar(): de manera aleatoria lanza un trigger de attack de los dos existentes y cambia el estado a Retirandose. En caso de que la distancia sea superior a el radio de ataque cambia de estado a Persiguiendo.

```
void Atacar()
{
    if (Random.value > 0.5f)
        animator.SetTrigger("attack1");
    else
        animator.SetTrigger("attack2");

    tiempoUltimoAtaque = Time.time;

    CambiarEstado(EstadoAbeja.Retirandose);
    timerRetirada = tiempoRetirada;
}
```

Ilustración 81. Atacar

Si el estado es Retirandose se ejecuta Retirarse(): se aleja del jugador y pasado un tiempo cambia al estado de Persiguiendo.

```
void Retirarse()
{
    Vector2 direccion = (rb.position - (Vector2)jugador.position).normalized;

    Vector2 destino = rb.position + direccion * distanciaRetirada;

    MoverConEvasion(destino);

    timerRetirada -= Time.fixedDeltaTime;

    if (timerRetirada <= 0)
    {
        CambiarEstado(EstadoAbeja.Persiguiendo);
    }

    Girar(jugador.position.x);
}
```

Ilustración 82. Retirarse

La función CambiarEstado() pone a false el booleano isMoving si estado cambia a Patrullando y pone a true isMoving si estado cambia a Persiguiendo.

```
void CambiarEstado(EstadoAbeja nuevoEstado)
{
    if (estadoActual == nuevoEstado) return;

    estadoActual = nuevoEstado;

    switch (estadoActual)
    {
        case EstadoAbeja.Patrullando:
            animator.SetBool("isMoving", false);
            break;

        case EstadoAbeja.Persiguiendo:
            animator.SetBool("isMoving", true);
            break;
    }
}
```

Ilustración 83. CambiarEstado

6. Assets utilizados

Para la tipografía se ha utilzado la fuente BoldPixels 2D de Unity Store: https://assetstore.unity.com/packages/2d/fonts/boldpixels-font-332078?srsltid=AfmBOorK4hVpW-BvxRG4X3OtfnqoN92-4G7HqZuu5c_nwGdIvs1q_ysk

Para terreno, fondo, interactuables, enemigos y jugador, se han utilizado assets gratuitos de itch.io:

Jugador: https://pixelfrog-assets.itch.io/pixel-adventure-1

Enemigos e interactuables: https://kevins-moms-house.itch.io/four-seasons-platformer-sprites

Terreno: https://cainos.itch.io/pixel-art-platformer-village-props

Fondo parallax: https://bongseng.itch.io/parallax-forest-desert-sky-moon

7. Bibliografía

Tutoriales menús y enemigos: https://www.youtube.com/@BravePixelG/videos

Tutorial personaje: https://www.youtube.com/watch?v=GbmRt0wydQU

Tutorial fondo parallax: https://www.youtube.com/watch?v=HnW0ugK1EGs